

# Brain Document Ingestion

Staging integration quickstart. Upload documents → extract → obligations → Q&A.

## Staging URLs

| Endpoint     | URL                                                                                   | Status |
|--------------|---------------------------------------------------------------------------------------|--------|
| API base     | <a href="https://staging-api.brain.fi/v1">https://staging-api.brain.fi/v1</a>         | Ready  |
| Health check | <a href="https://staging-api.brain.fi/health">https://staging-api.brain.fi/health</a> | Ready  |

## Authentication

Staging uses a demo token flow — no email signup needed. Get a token in one request:

```
# Get a demo token (valid 24h, includes raw:read, raw:write, ledger:read, wil
curl -s https://staging-api.brain.fi/v1/demo/token \
  -H "Content-Type: application/json" \
  -d '{}' | jq .

# Response:
{
  "token": "eyJhbGciOiJIUzI1NiIs... ",
  "tenant_id": "tnt_abc123",
  "expires_at": "2026-07-10T12:00:00Z"
}
```

Use the `token` in the `Authorization: Bearer` header for all subsequent calls.

## The pipeline: 4 steps

## **POST** /v1/raw/ingest

Upload a file (PDF, CSV, XLSX, image, or scanned PDF).

```
curl -X POST https://staging-api.brain.fi/v1/raw/ingest \
  -H "Authorization: Bearer $TOKEN" \
  -F "source_type=pdf_upload" \
  -F "file=@invoice.pdf" \
  -F "mime_type=application/pdf" | jq .
```

```
# Response (201 Created):
{
  "raw_id": "raw_abc123",
  "sha256": "7f1e3a...",
  "deduplicated": false
}
```

Max 50 MB. Returns 200 if the identical file already exists (deduplicated: true). The file is stored and ready for extraction.

## **POST** /v1/raw/{raw\_id}/extract

Extract text and structured data from the uploaded file.

```
curl -X POST https://staging-api.brain.fi/v1/raw/raw_abc123/extract \
  -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" \
  -d '{}' | jq .
```

```
# Response (200 OK):
{
  "parsed_id": "parsed_xyz789",
  "confidence": 0.45
}
```

Idempotent: re-posting for the same file returns the existing result. Confidence  $\leq 0.5$  means the data is extracted but advisory-only (not authoritative for payments). The extraction service (agents) runs in the background; this endpoint triggers it.

## **GET** /v1/ledger/obligations

Read back the extracted obligations. Wait ~15 seconds after extraction before polling.

```
curl https://staging-api.brain.fi/v1/ledger/obligations \
-H "Authorization: Bearer $TOKEN" | jq .
```

```
# Response (200 OK):
```

```
{
  "obligations": [
    {
      "id": "obl_123",
      "type": "payable",
      "direction": "outbound",
      "amount_due": "1500.00",
      "currency": "USD",
      "due_date": "2026-08-15",
      "status": "upcoming",
      "counterparty_id": "cp_abc",
      "provenance": "agent_contributed",
      "confidence": 0.45
    }
  ]
}
```

Filter by status: `?status=upcoming|due|overdue` . All document-extracted items have confidence  $\leq 0.5$  and are `agent_contributed` — they're insights, not authoritative data.

**POST** [/v1/wiki/question](#)

Ask a plain-language question about the extracted obligations.

```
curl -X POST https://staging-api.brain.fi/v1/wiki/question \
-H "Authorization: Bearer $TOKEN" \
-H "Content-Type: application/json" \
-d '{ "question": "what is due this month" }' | jq .
```

```
# Response (200 OK):
```

```
{
  "answer": "You have 1 payable due this month: $1500 to Vendor Inc on 2026-08-15",
  "evidence": [
    {
      "obligation_id": "obl_123",
      "amount": "1500.00",
      "counterparty": "Vendor Inc",

```

```
    "due_date": "2026-08-15"  
  }  
]  
}
```

The answer is grounded in the obligations above, with the evidence rows it used.

## What's extracted

The extraction engine (a Python agent) runs five deterministic + LLM passes:

- **CSV / XLSX**: Tabular cells are parsed directly.
- **Digital PDF**: Text layer is extracted as-is.
- **Scanned image or image-only PDF**: Vision model (OpenAI gpt-4o) extracts text, then LLM fields.
- **LLM field extraction**: vendor name, amount, date, line items, payment terms, memo — whatever the invoice contains.
- **Confidence capping**: All document reads are capped at confidence  $\leq 0.5$  and marked `agent_contributed`. This is advisory data, not authoritative. A payment authorization requires corroboration or an explicit human confirm.

## Timing expectations

| Step                             | Time                                      |
|----------------------------------|-------------------------------------------|
| POST /raw/ingest (upload)        | < 1s                                      |
| POST /raw/{id}/extract (trigger) | < 1s response; 5–15s to process           |
| GET /ledger/obligations (poll)   | Immediate once ready (~15s after extract) |
| POST /wiki/question (ask)        | 1–3s                                      |

Wait at least 15 seconds after calling extract before polling obligations. If you see 0 obligations on the first poll, wait another 5–10 and try again.

# Example integration flow (in pseudocode)

```
// 1. Get a demo token
token = POST /demo/token → .token

// 2. Upload a document (e.g. from the Replit UI file picker)
raw_id = POST /raw/ingest (file, auth: token) → .raw_id

// 3. Trigger extraction
parsed_id = POST /raw/{raw_id}/extract (auth: token) → .parsed_id

// 4. Poll for obligations (wait 15s, then poll every 2–5s until you get results)
obligations = GET /ledger/obligations (auth: token) → .obligations
// Extract the items and render them in the Replit UI

// 5. Optional: ask a question
answer = POST /wiki/question ({ question: "...", auth: token } → .answer
// Show the answer to the user
```

## Error handling

| Status      | Scenario                                                   | Handling                                                               |
|-------------|------------------------------------------------------------|------------------------------------------------------------------------|
| 201,<br>200 | Success                                                    | Proceed to the next step.                                              |
| 400         | Invalid request (bad JSON, missing field)                  | Check your request body and headers.                                   |
| 401         | Missing or expired token                                   | Get a new token via /demo/token.                                       |
| 404         | File not found (invalid raw_id, cross-tenant access)       | Check the raw_id is correct and matches your token's tenant.           |
| 422         | Extraction failed (unsupported file type, OCR unavailable) | Try a different file. Supported: PDF, CSV, XLSX, images, scanned PDFs. |

|     |                                |                                                                                                         |
|-----|--------------------------------|---------------------------------------------------------------------------------------------------------|
| 501 | Extraction service unavailable | Transient. Retry in a few seconds.                                                                      |
| 5xx | Server error                   | Check the staging /health endpoint. File a bug with the raw_id and request ID from the response header. |

## Known limitations

**Staging only:** The demo token path exists for testing. Real users on production will use email signup/login (coming soon). For now, staging is the development environment.

- **No real email delivery yet** — production signup verification is staged but not wired. Staging uses demo tokens.
- **Confidence cap** — All extracted data is at confidence  $\leq 0.5$ . This is by design. Document reads cannot move money alone; they must be corroborated or confirmed.
- **No real counterparty matching** — Vendor names are extracted as text only. Matching to your internal vendor database is a platform responsibility.
- **OCR has limits** — Handwritten, skewed, or low-quality scans may fail. The API returns a 422 if OCR produces no usable text.

## Next steps

1. **Copy the URLs and token flow above** into your integration code.
2. **Test with a sample PDF or CSV** (use the curl examples, or build your own HTTP client).
3. **Render the obligations in your UI** — show the extracted data as insights, label them advisory, never as money-ready without confirmation.
4. **Set up a "questions" widget** — use POST /wiki/question to let users ask free-form questions about their obligations.
5. **When ready for production**, we'll flip your platform to use <https://api.brain.fi/v1> and real email signup/login. The API contracts stay the same.

**Support:** File issues with staging behavior, include the request/response and raw\_id if applicable.

**Staging API health:** <https://staging-api.brain.fi/health>